You only need to change how the data is assigned in the script block. You can no longer serialize a form because there isn't one, so create an object with properties that mirror the input model the page handler expects as a parameter. You only need to specify property values required by the page handler. The following listing shows the altered button click event handler.

Listing 11.14 Altering the button click event handler to use an object

```
$('.btn-success').on('click', function(){
    const data = {                                          ❶
            startdate: $('#StartDate').val(),               ❷
            enddate: $('#EndDate').val(),                   ❷
            numberofguests: $('#NumberOfGuests').val(),     ❷
            __requestverificationtoken:
$('[name="__RequestVerificationToken"]').val(),             ❸
            property:{                                      ❹
                dayrate: $('#Property_DayRate').val()       ❹
            }                                               ❹
        }
    $.post('?handler=booking', data, function(totalCost){
        alert(`Your stay will cost ${totalCost}`);
    });
});
```

❶ Create an object.

❷ Use the jQuery val() method to get the form field values and assign them to properties.

❸ The token's hidden field doesn't have an id attribute, so you use its name as the jQuery selector.

❹ The Property property of the input model is nested.

The jQuery library takes care of converting the JavaScript object represented by the data variable into a URL-encoded string for posting and assigning the correct content-type header to the request (`x-www-form-urlencoded`). In the plain JavaScript version of the button click handler, which is available in the chapter downloads, you flatten the input model by including a top-level `DayRate` property. This is so you can use the browser's `URLSearchParams` interface (http://mng.bz/49vj) to